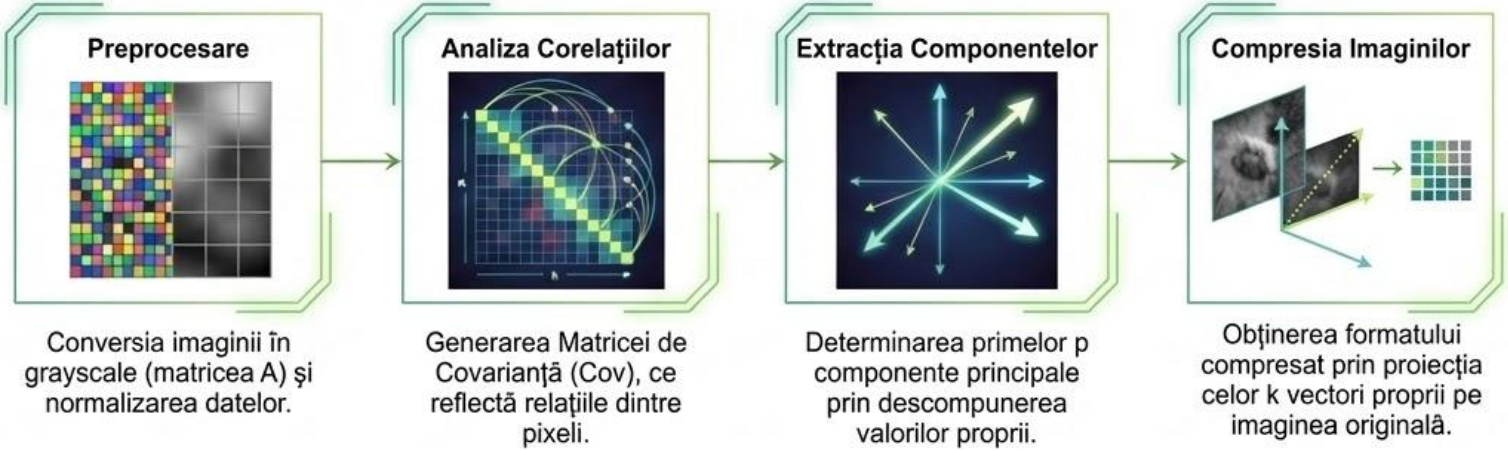
Compresia imaginilor folosind PCA

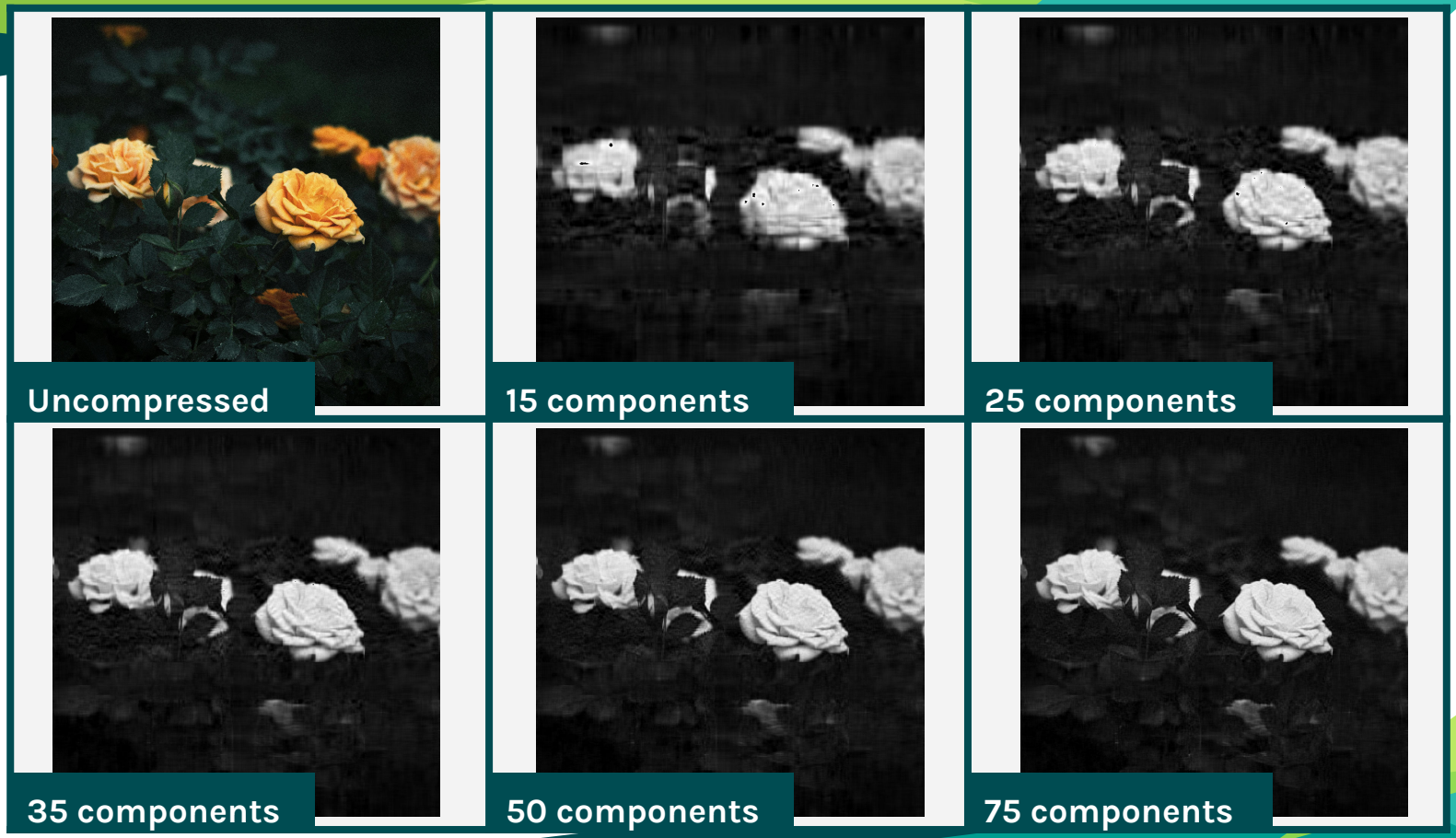
Echipă:
Georgescu Andrei, 342C1
Lache Alexandra, 342C1

Descrierea proiectului

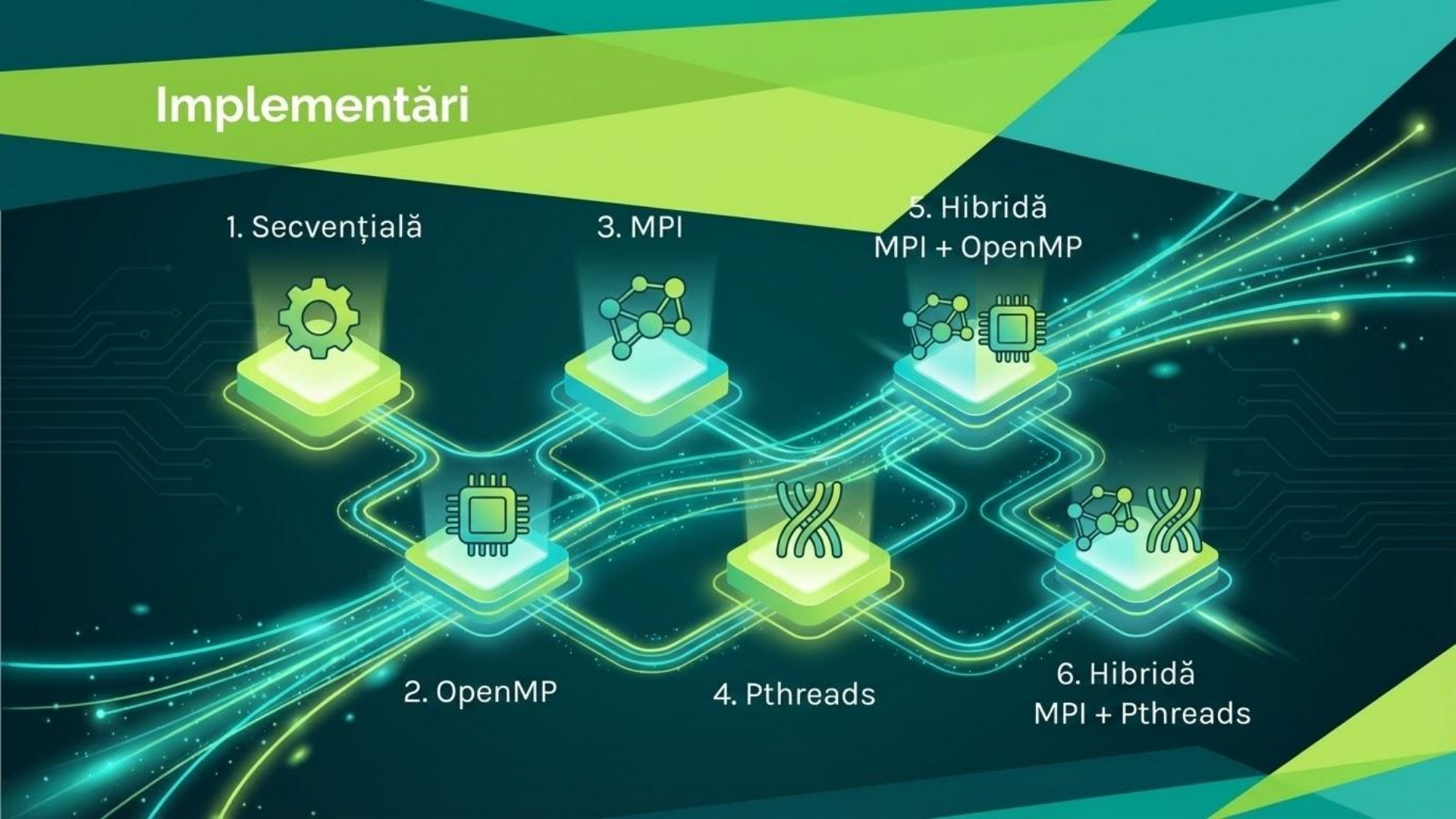
I. **Principiu**: Reducerea dimensiunii datelor de la n la k (k<n) prin eliminarea dependențelor liniare între pixeli.
II. **Complexitate**: temporală - O(k * n^3)
III. **Etapele procesului**:



Rezultate Compresie și Reconstrucție



Implementări



Implementare Secvențială: Evoluție și Optimizare

Provocare: Calculul valorilor proprii pentru matricea de covarianță.

I. Eigen

- Utilizarea solverului de baza din biblioteca pentru descompunerea completa.
- **Limitare**: Consum excesiv de memorie, intrucat se calculeaza toate valorile și vectorii proprii.
- **Rezultat**: crash pentru imagini mai mari de 375x375 pixeli.

II. Eigen + Spectra

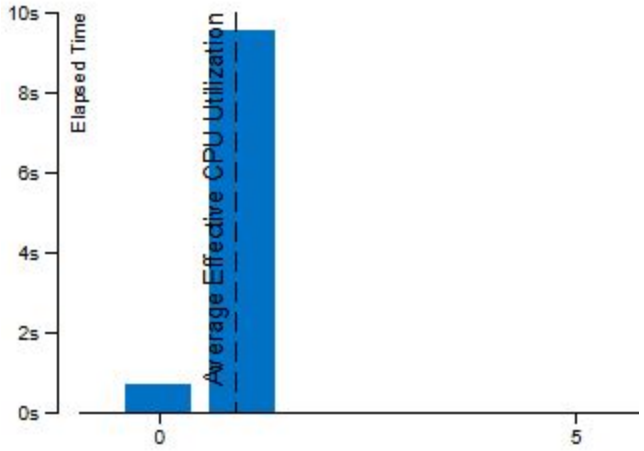
- Utilizarea unui solver specializat pentru a extrage doar cele mai importante k componente principale.
- **Limitare**: “Black-box”. Profilingul a identificat ca majoritatea timpului este consumat de funcții interne ale librariei.
- **Rezultat**: dificil de paralelizat.

III. Algoritmul lui Lanczos

- Implementare proprie pentru construcția iterativă a matricei tridiagonale T și aflarea valorilor proprii ale acesteia folosind Eigen.
- **Rezultat**: control total asupra implementării și memoriei, baza ideala pentru paralelizarea ulterioară.

Profiling Varianta Secvențială

Function	Module	CPU Time	% of CPU Time
multiply_matrices	seq_pca_vtune	9.400s	98.6%
transpose_matrix	seq_pca_vtune	0.060s	0.6%
convert_rgb_to_grayscale	seq_pca_vtune	0.020s	0.2%
lanczos_algorithm	seq_pca_vtune	0.010s	0.1%
decompress_pca	seq_pca_vtune	0.010s	0.1%
[Others]	N/A*	0.030s	0.3%



Implementari Paralele

OpenMP

- Paralelizarea buclelor interne mari (înmulțiri de matrici, produse scalare) dependente de dimensiunea N.
- S-a optat pentru scheduling static, deoarece sarcina de lucru este uniform distribuită.

Pthreads

- Utilizarea unui Thread Pool pentru a elimina overhead-ul creat de crearea/distrugerea repetată a thread-urilor.
- Thread-ul principal coordonează execuția. Thread-urile workeri semnaleaza ca executia a fost incheiata prin intermediul semafoarelor.

Profiling OMP

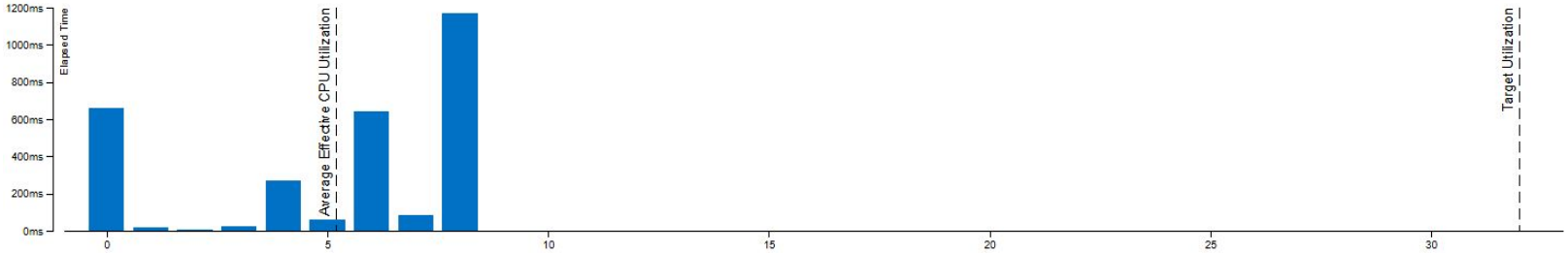
Function	Module	CPU Time	% of CPU Time
multiply_matrices._omp_fn.5	omp_pca_vtune	11.550s	96.1%
func@0x1dfd4	libgomp.so.1	0.210s	1.7%
transpose_matrix._omp_fn.4	omp_pca_vtune	0.110s	0.9%
func@0x1de24	libgomp.so.1	0.070s	0.6%
multiply_matrices	omp_pca_vtune	0.020s	0.2%
[Others]	N/A*	0.060s	0.5%

4 Threaduri



Profiling OMP

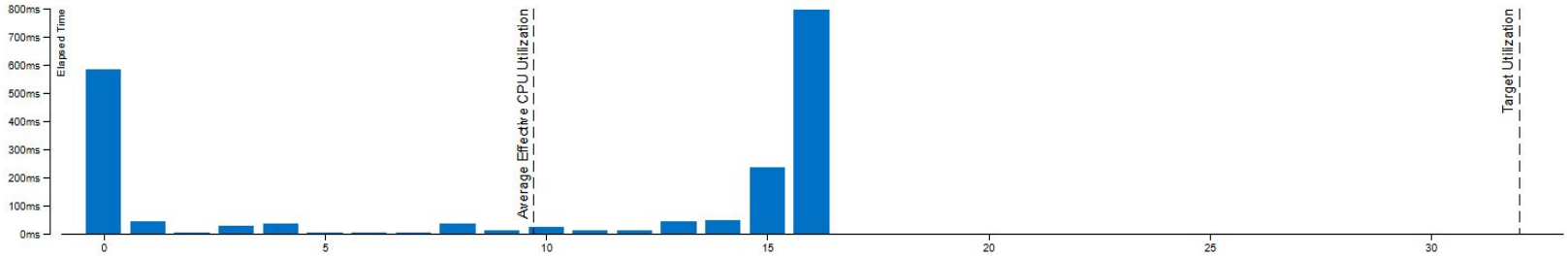
Function	Module	CPU Time	% of CPU Time
multiply_matrices._omp_fn.5	omp_pca_vtune	14.569s	95.8%
func@0x1dfd4	libgomp.so.1	0.159s	1.0%
func@0x1de24	libgomp.so.1	0.131s	0.9%
transpose_matrix._omp_fn.4	omp_pca_vtune	0.130s	0.9%
normalize_matrix._omp_fn.3	omp_pca_vtune	0.040s	0.3%
[Others]	N/A*	0.171s	1.1%



8 Threaduri

Profiling OMP

Function	Module	CPU Time	% of CPU Time
multiply_matrices._omp_fn.5	omp_pca_vtune	17.550s	93.8%
transpose_matrix._omp_fn.4	omp_pca_vtune	0.330s	1.8%
func@0x1de24	libgomp.so.1	0.260s	1.4%
func@0x1dfd4	libgomp.so.1	0.250s	1.3%
compute_dot_product._omp_fn.7	omp_pca_vtune	0.060s	0.3%
[Others]	N/A*	0.250s	1.3%

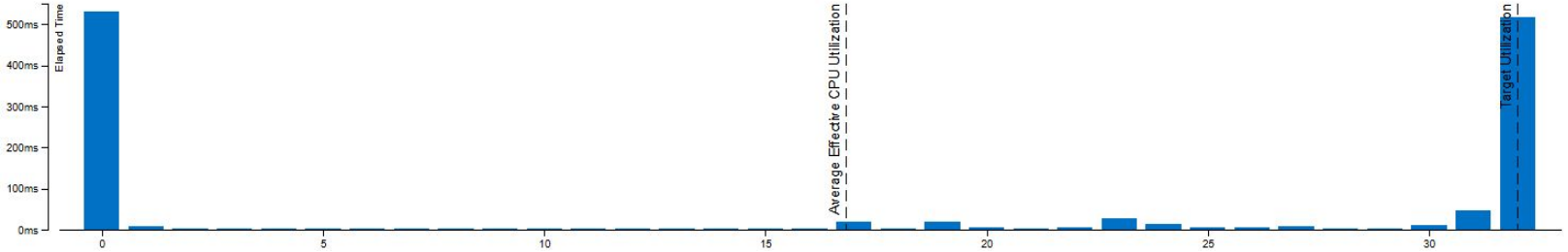


16 Threaduri

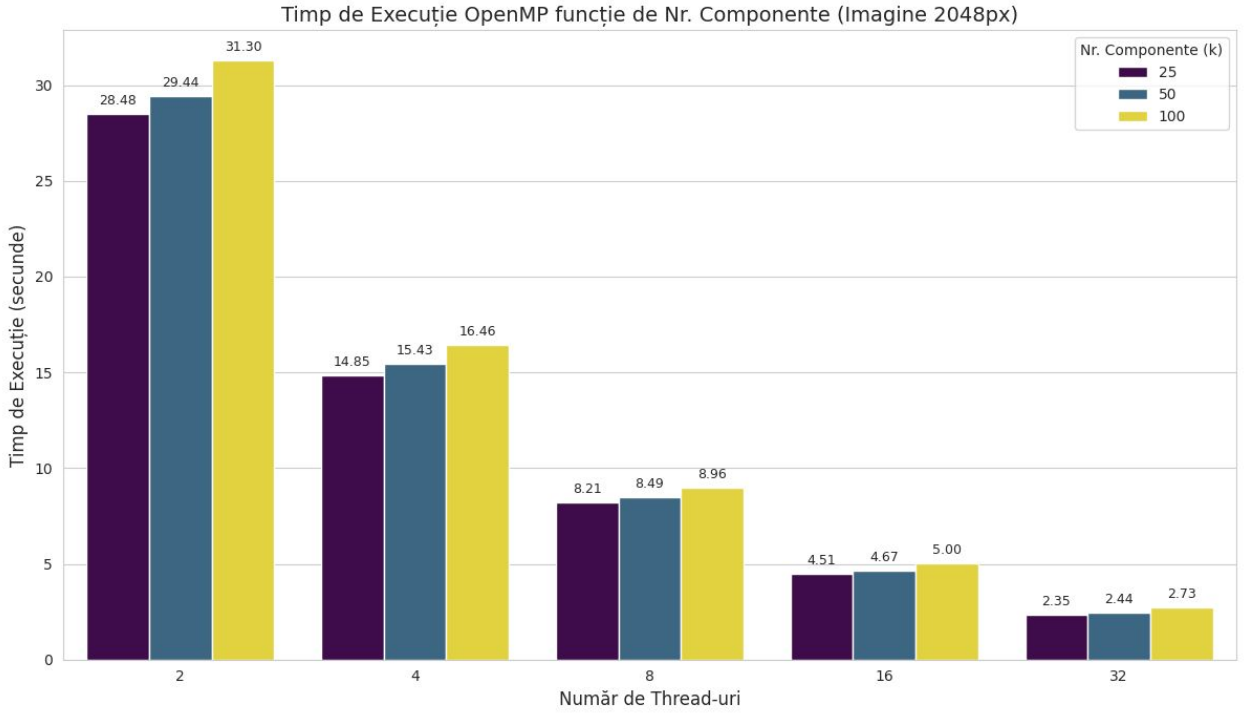
Profiling OMP

Function	Module	CPU Time	% of CPU Time
multiply_matrices._omp_fn.5	omp_pca_vtune	18.310s	87.8%
func@0x1de24	libgomp.so.1	0.799s	3.8%
transpose_matrix._omp_fn.4	omp_pca_vtune	0.711s	3.4%
func@0x1dfd4	libgomp.so.1	0.540s	2.6%
compute_dot_product._omp_fn.7	omp_pca_vtune	0.170s	0.8%
[Others]	N/A*	0.329s	1.6%

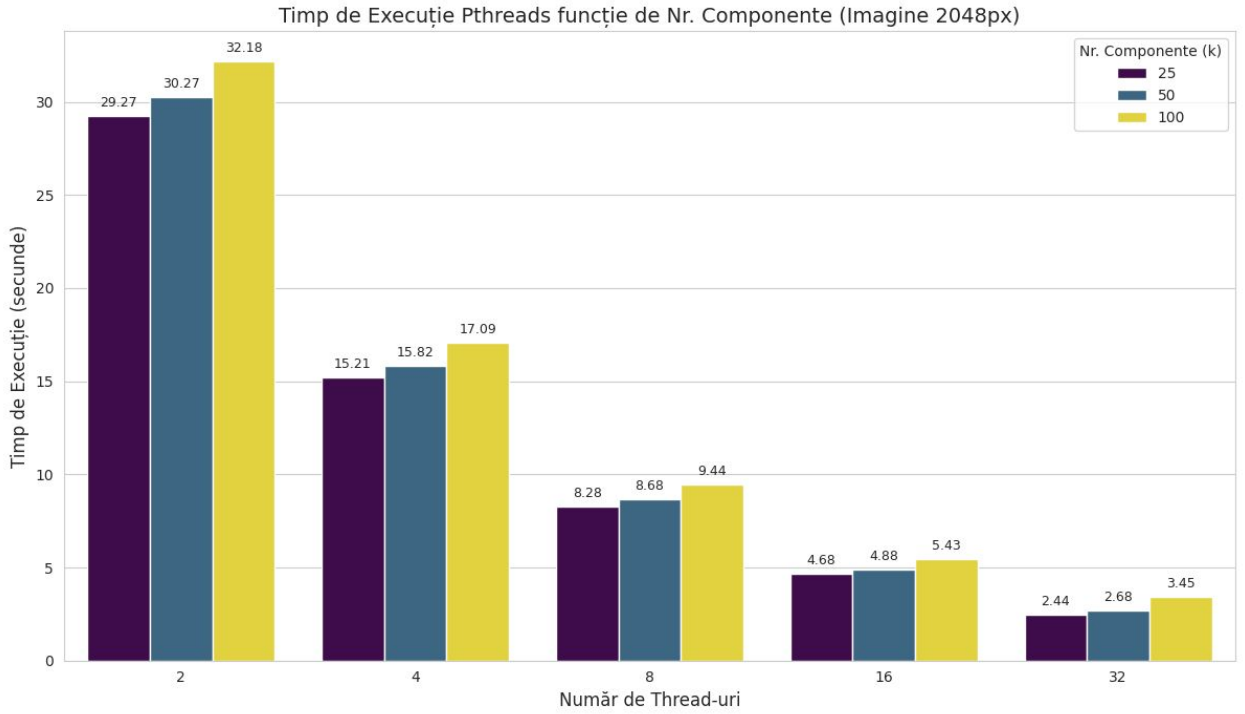
32 Threaduri



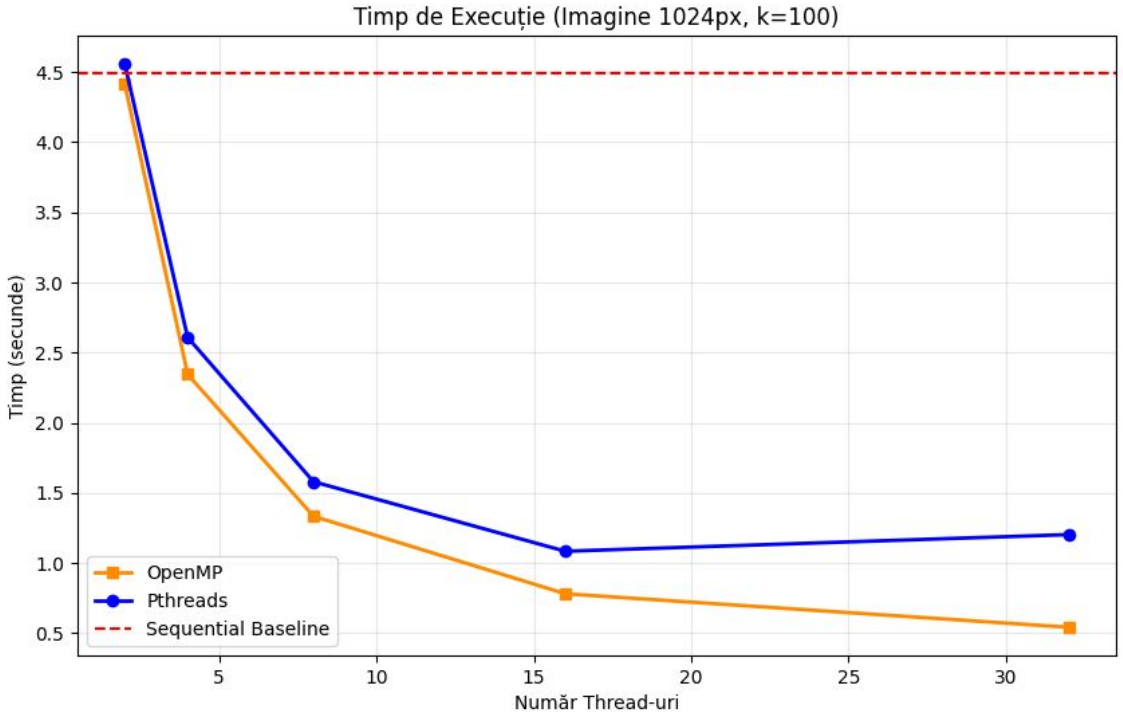
Rezultate Pthreads



Rezultate OpenMP



Pthreads vs OpenMP

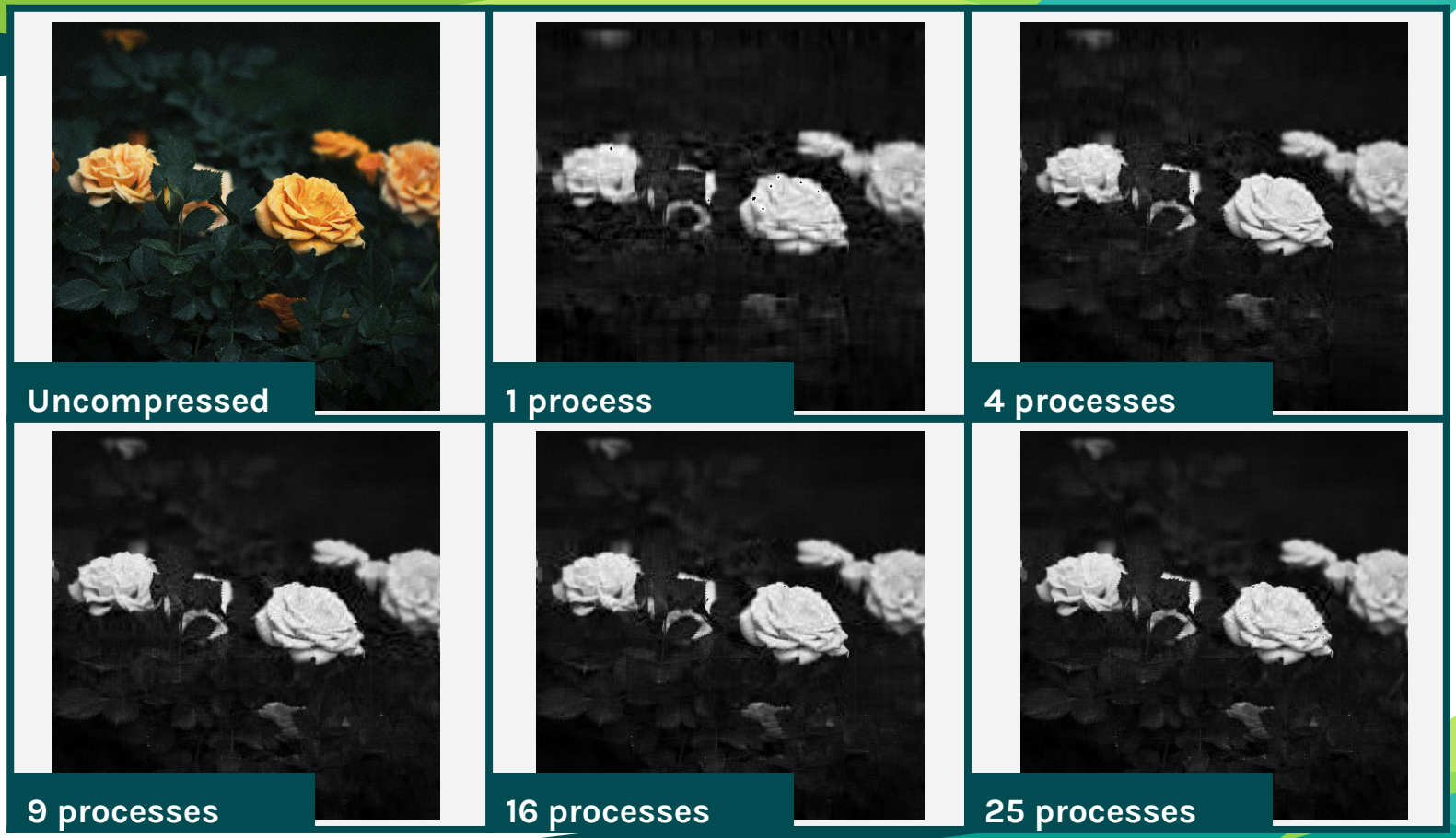


Implementarea distribuită

MPI

Paralelizeaza efortul de compresie prin împărțirea imaginilor în mai multe sub-imagini. Se alege un număr de sub-imagini de forma k^2, pentru a împărții ușor imaginea. Root-ul partiționează imaginea către k coordonatori, care la rândul lor partiționează către k coordonați. Fiecare sub-imagine este tratată de câte un proces care rulează independent algoritmul de compresie, apoi întoarce rezultatul spre coordonatori, care la rândul lor întorc spre Root.

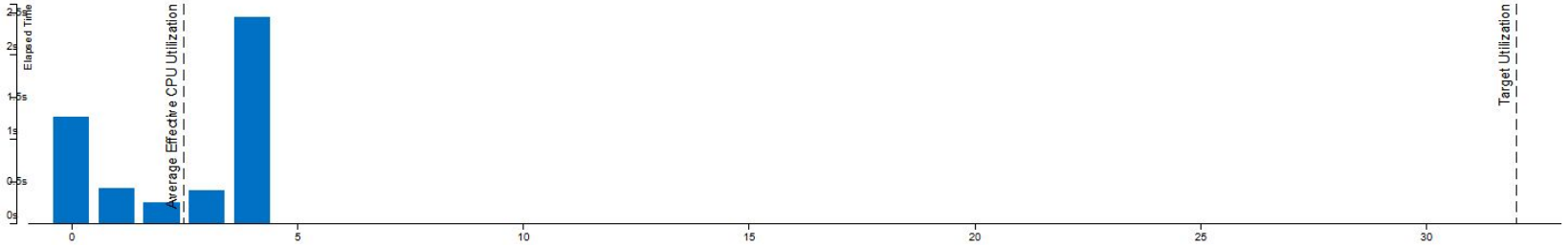
Rezultate Compresie și Reconstrucție Distribuită



Profiling MPI

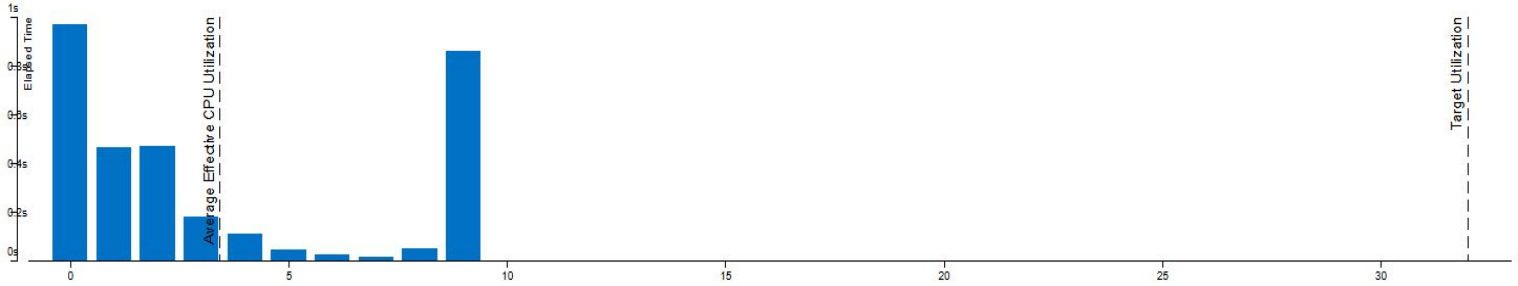
Function	Module	CPU Time	% of CPU Time
multiply_matrices	mpi_fep_pca_vtune	9.190s	77.6%
MPI_Bcast	libmpi.so.40	0.810s	6.8%
MPI_Init	libmpi.so.40	0.419s	3.5%
OS_BARESYSCALL_DoCallAsmIntel64Linux	libc-dynamic.so	0.190s	1.6%
ompi_mpi_finalize	libmpi.so.40	0.170s	1.4%
[Others]	N/A*	1.061s	9.0%

4 Procese



Profiling MPI

Function	Module	CPU Time	% of CPU Time
multiply_matrices	mpi_fep_pca_vtune	6.699s	61.0%
MPI_Init	libmpi.so.40	1.140s	10.4%
MPI_Gatherv	libmpi.so.40	0.440s	4.0%
[ld-linux-x86-64.so.2]	ld-linux-x86-64.so.2	0.400s	3.6%
MPI_Bcast	libmpi.so.40	0.370s	3.4%
[Others]	N/A*	1.931s	17.6%

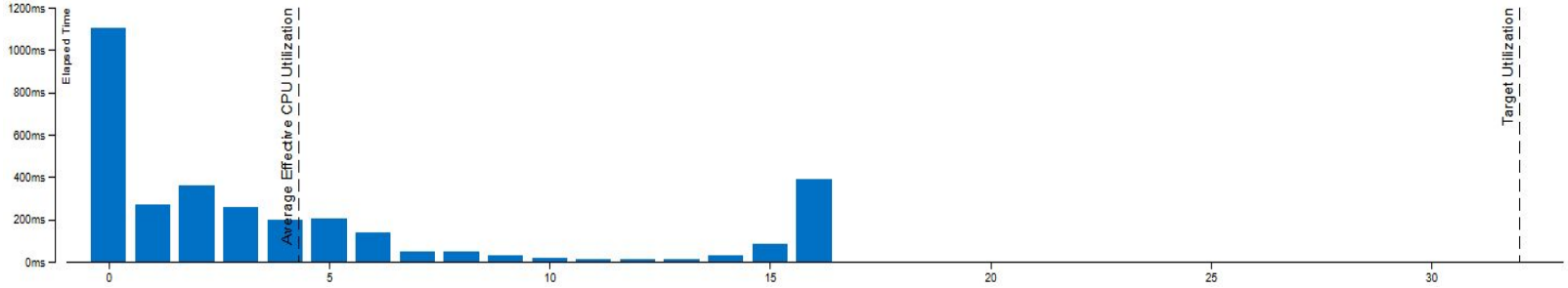


9 Procese

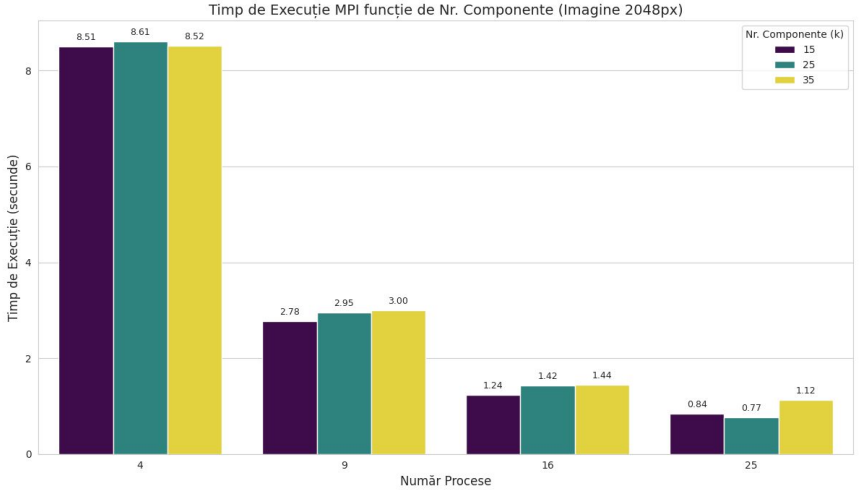
Profiling MPI

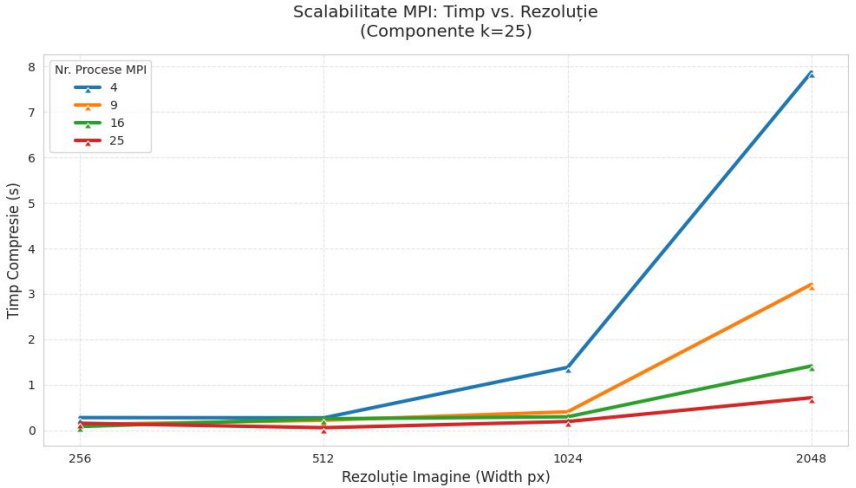
Function	Module	CPU Time	% of CPU Time
multiply_matrices	mpi_fep_pca_vtune	5.760s	41.5%
MPI_Init	libmpi.so.40	2.381s	17.2%
[ld-linux-x86-64.so.2]	ld-linux-x86-64.so.2	0.840s	6.1%
ompi_mpi_finalize	libmpi.so.40	0.780s	5.6%
MPI_Bcast	libmpi.so.40	0.770s	5.5%
[Others]	N/A*	3.349s	24.1%

16 Procese



Rezultate MPI





Implementari hibride

MPI + Pthreads/OpenMP

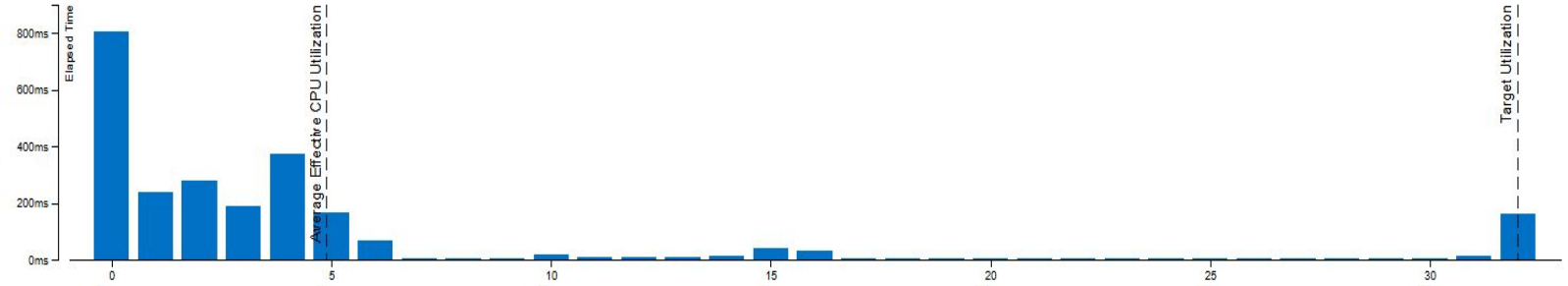
Implementările hibride se bazează pe combinarea abordărilor prezentate anterior, folosind multiprocessing cu MPI pentru a partiționa imaginea apoi optimizând algoritmul de compresie prin multithreading cu ajutorul Pthreads/OpenMP.

Profiling MPI-OMP

Function	Module	CPU Time	% of CPU Time
multiply_matrices._omp_fn.5	hybrid_fep_pca_vtune	5.735s	47.4%
MPI_Init	libmpi.so.40	1.989s	16.5%
OS_BARESYSCALL_DoCallAsmIntel64Linux	libc-dynamic.so	0.651s	5.4%
ompi_mpi_finalize	libmpi.so.40	0.540s	4.5%
[ld-linux-x86-64.so.2]	ld-linux-x86-64.so.2	0.500s	4.1%
[Others]	N/A*	2.675s	22.1%

16 Procese

2 Threaduri

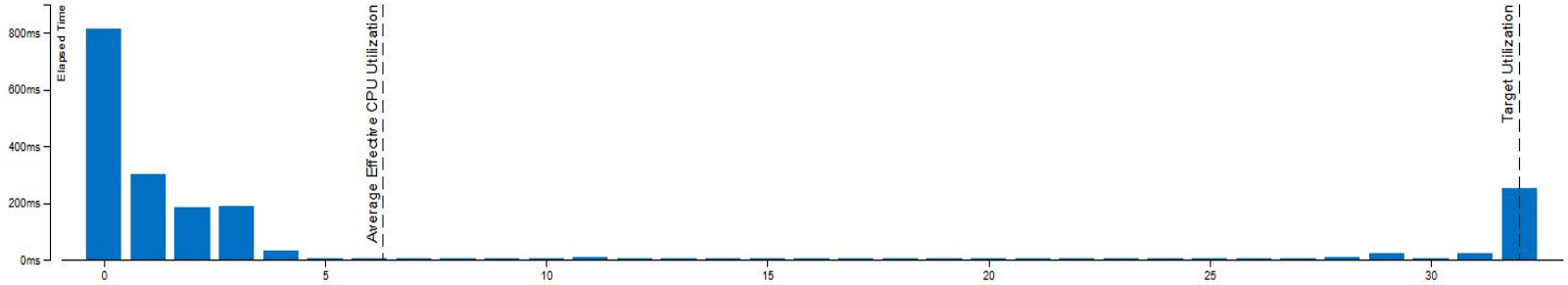


Profiling MPI-OMP

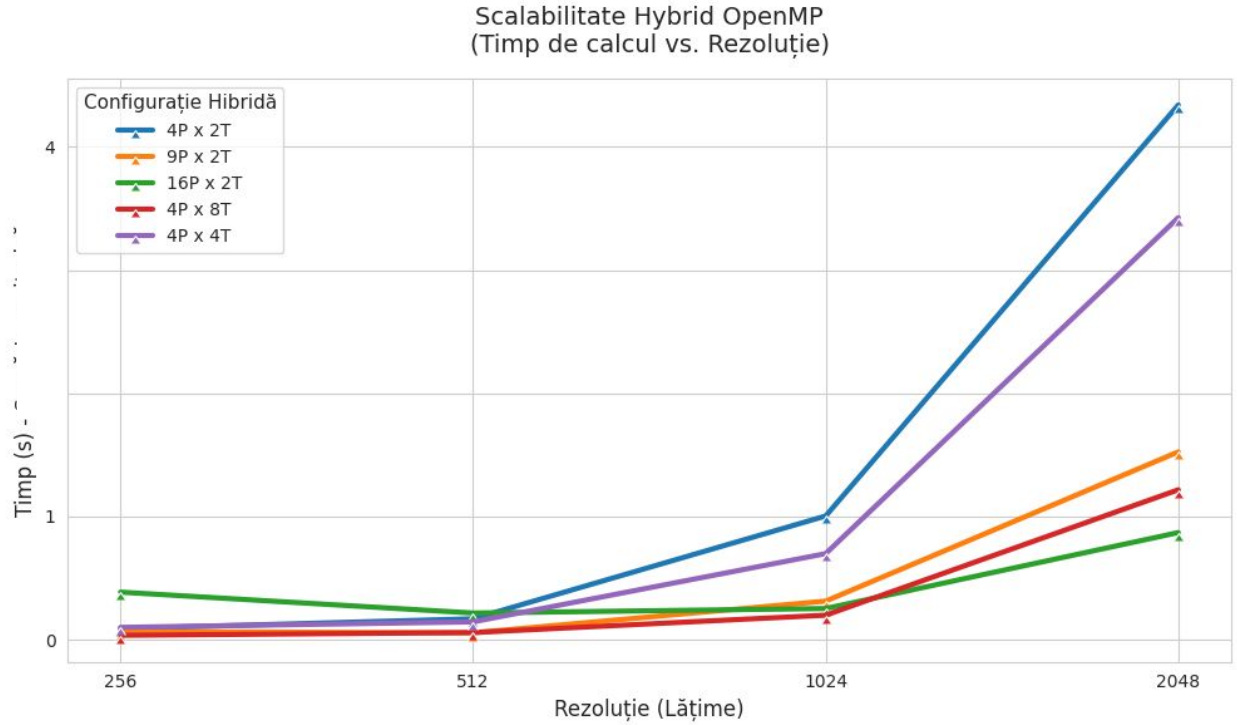
Function	Module	CPU Time	% of CPU Time
multiply_matrices._omp_fn.5	hybrid_fep_pca_vtune	9.770s	82.7%
MPI_Bcast	libmpi.so.40	0.410s	3.5%
MPI_Init	libmpi.so.40	0.271s	2.3%
func@0x1de24	libgomp.so.1	0.260s	2.2%
[ld-linux-x86-64.so.2]	ld-linux-x86-64.so.2	0.160s	1.4%
[Others]	N/A*	0.949s	8.0%

4 Procese

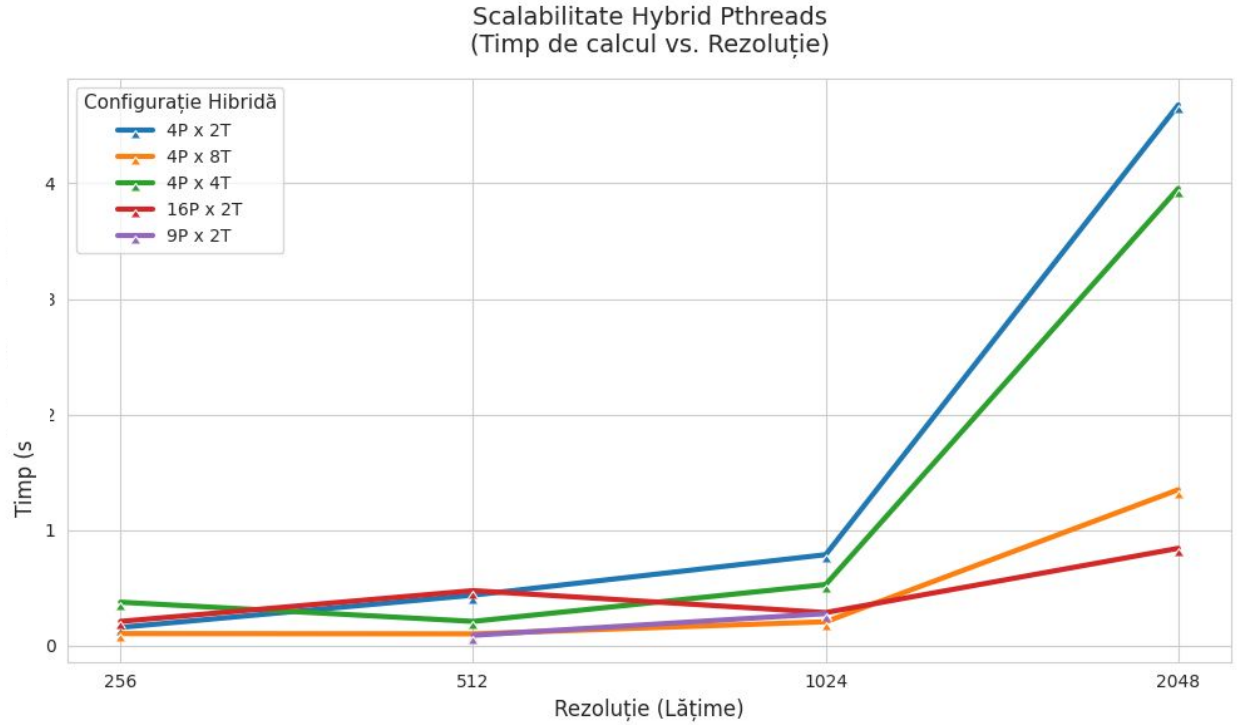
8 Threaduri



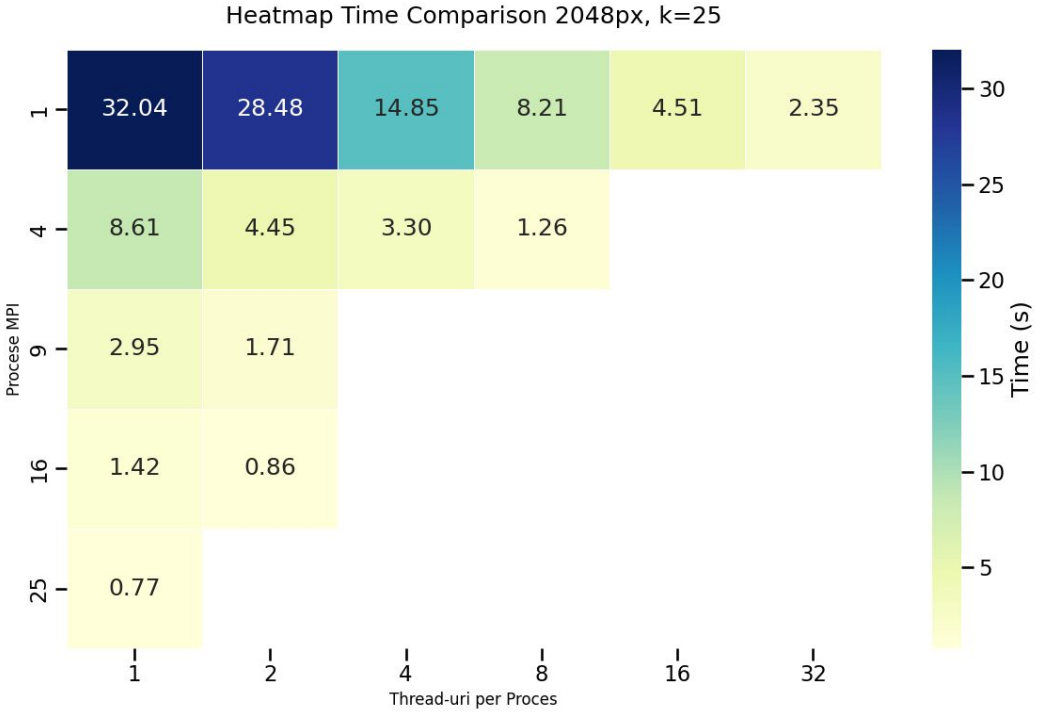
MPI+OpenMP


Scalabilitate Hybrid OpenMP
(Timp de calcul vs. Rezoluție)
Configurație Hibridă
4P x 2T
9P x 2T
16P x 2T
4P x 8T
4P x 4T
Timp (s)
4
1
0
256
512
1024
2048
Rezoluție (Lățime)

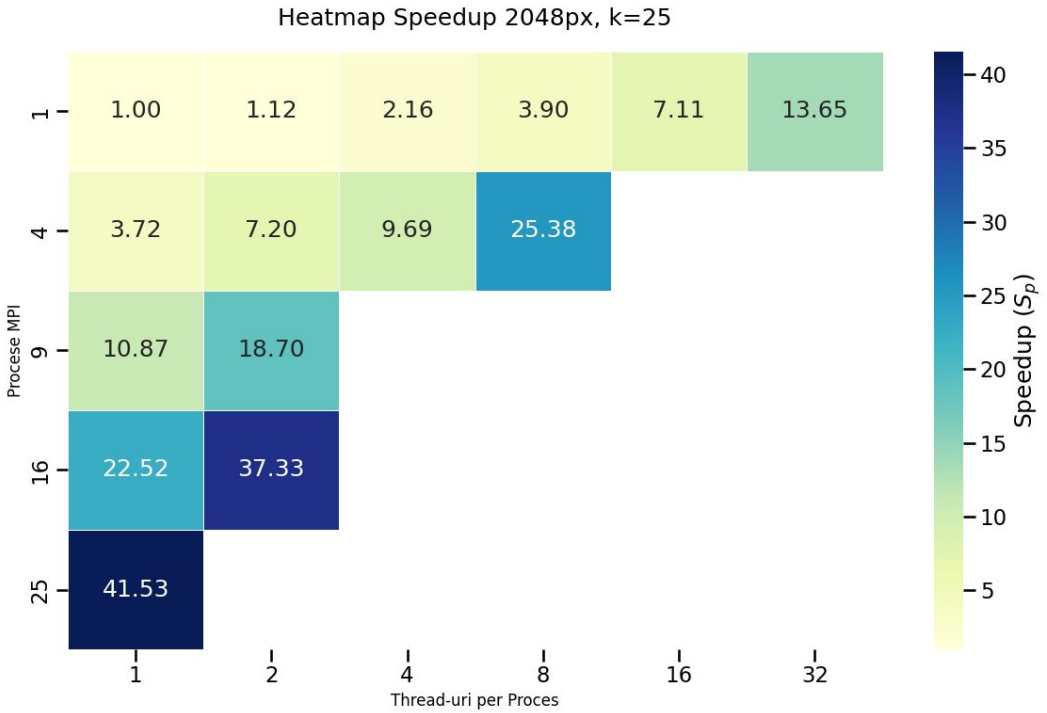
MPI+Pthreads



Heatmap Timp



Heatmap Speedup



Concluzii

Rulările pur MPI cu ***25 de procese*** sunt cele cu cea mai bună performanță în timp.

Acest lucru este parțial datorat unei rate de *cache miss* mai mică precum și a reducerii dimensionalității înmulțirii, ambele cauzate de partiționarea imaginilor în sub-imagini. O performanță similară este de asemenea atinsă și în cazul **16Px2T**, pe considerente similare. Scalabilitatea la un număr mai mare de CPU-uri totuși probabil că ar favoriza metode hibride, deoarece metodele pur MPI rulează optim doar atunci când folosim k^2 CPU-uri.